

Lab 8: Accelerated Proximal Methods

Due Wednesday, April 8, 11:59pm

In Lab 7, you implemented gradient descent to solve imaging inverse problems. Gradient descent is a powerful technique, but we often want to add additional constraints to our optimization objective. In this lab, we introduce proximal methods that enforce constraints like non-negativity, and an accelerated variant called FISTA that converges faster. We will then use these tools to explore a native sparsity prior.

In practice, we can combine multiple proximal operators using a proximal average:

$$\frac{1}{m} \sum_{i=1}^m \text{prox}_{f_i} = \text{prox}_g$$

It is common to use a non-negativity prior in addition to another sparsity prior. For example, to use both non-negativity and native sparsity together, you can do:

$$\text{prox}_g = \frac{1}{2} (\max(\mathbf{x}, 0) + \text{soft}(\mathbf{x}, \lambda/L))$$

In this lab you will use FISTA to solve the following optimization problems:

$$\text{non-negativity prior: } L(\mathbf{x}) = \frac{1}{2} \|\mathbf{y} - \mathbf{Ax}\|_2^2 + \text{non-neg}(\mathbf{x}) \quad (1)$$

$$\text{native prior: } L(\mathbf{x}) = \frac{1}{2} \|\mathbf{y} - \mathbf{Ax}\|_2^2 + \lambda \|\mathbf{x}\|_1 \quad (2)$$

Allowed libraries. You may use `numpy`, `scipy`, `matplotlib`, and `imageio`.

Starter code. We provide `priors.py`, which contains the proximal operator functions you will implement. `prox_nonneg` is already provided as a reference. Fill in `soft_thresh` and `prox_native` (marked `TODO`). You will also need your completed `inverse_solver.py` from Lab 7.

Note: `priors.py` also contains wavelet and TV helpers that you will use in Lab 9. You can ignore those for now.

Each proximal operator follows the signature `prox_fn(x, lam, step.size)` and can be passed directly to `InverseSolver`:

```
from inverse_solver import InverseSolver
from priors import prox_native

solver = InverseSolver(psf, algorithm='fista',
                      prox_fn=prox_native, lam=1e-2)
x_hat = solver(y)
```

Your data. Use your own DiffuserCam image (captured in Lab 6) for all problems. You may use the provided sample data for debugging, but your submission must show results on your own capture.

1 Proximal Methods

One common constraint is non-negativity, which states that pixel values must be non-negative. This is a really useful constraint in imaging systems because sensors always capture positive intensity values.

Non-negativity can be expressed as:

$$g(\mathbf{x}) = \begin{cases} 0 & \text{if } \mathbf{x} \geq 0 \\ \infty & \text{otherwise} \end{cases}$$

If we include this constraint in our optimization objective, we have:

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + g(\mathbf{x})$$

While we can take the gradient of the first term, the second term $g(\mathbf{x})$ is not differentiable at 0, so we cannot as easily use a simple gradient descent method. Luckily, there exist some mathematical tricks, called *proximal methods*, that we can use to solve these sorts of problems. The proximal gradient method has the form:

Input: $\mathbf{x}_0 \in \mathbb{R}^n$ and $L \geq \lambda_{\max}(\mathbf{A}^* \mathbf{A})$

while $\mathbf{x}_k$ not converged ($k = 1, 2, \dots$) **do**

$$\mathbf{w}_k \leftarrow \mathbf{x}_k - \frac{1}{L} \mathbf{A}^* (\mathbf{A} \mathbf{x}_k - \mathbf{y})$$

$$\mathbf{x}_{k+1} \leftarrow \text{prox}_{g/L}(\mathbf{w}_k)$$

end while

Output: $\mathbf{x}_* \leftarrow \mathbf{x}_k$

Where prox is a proximal operator. The proximal operator of non-negativity becomes: $\text{prox}_{g/L} = \max(\mathbf{x}, 0)$. We will call this proximal-gradient descent method ISTA with a non-negativity constraint.

In `InverseSolver`, proximal operators are passed as the `prox_fn` argument. For non-negativity:

```
def prox_nonneg(x, lam, step_size):
    return np.maximum(x, 0)

solver = InverseSolver(psf, algorithm='ista',
                      prox_fn=prox_nonneg)
```

Accelerated proximal method (FISTA).

To make our algorithm converge a bit faster, we can use a trick based on Nesterov acceleration to speed up the algorithm. This accelerated algorithm is called accelerated proximal method, or fast-ISTA (FISTA). This algorithm is given by:

Input: $\mathbf{x}_0 \in \mathbb{R}^n$, $\mathbf{p}_1 \leftarrow \mathbf{x}_0$, $t_1 \leftarrow 1$, and $L \geq \lambda_{\max}(\mathbf{A}^* \mathbf{A})$

while $\mathbf{x}_k$ not converged ($k = 1, 2, \dots$) **do**

$$\mathbf{w}_k \leftarrow \mathbf{p}_k - \frac{1}{L} \mathbf{A}^* (\mathbf{A} \mathbf{p}_k - \mathbf{y})$$

$$\mathbf{x}_{k+1} \leftarrow \text{prox}_{g/L}(\mathbf{w}_k)$$

$$t_{k+1} \leftarrow \frac{1 + \sqrt{1 + 4t_k^2}}{2}; \quad \beta_{k+1} \leftarrow \frac{t_k - 1}{t_{k+1}}$$

$$\mathbf{p}_{k+1} \leftarrow \mathbf{x}_{k+1} + \beta_{k+1}(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

end while

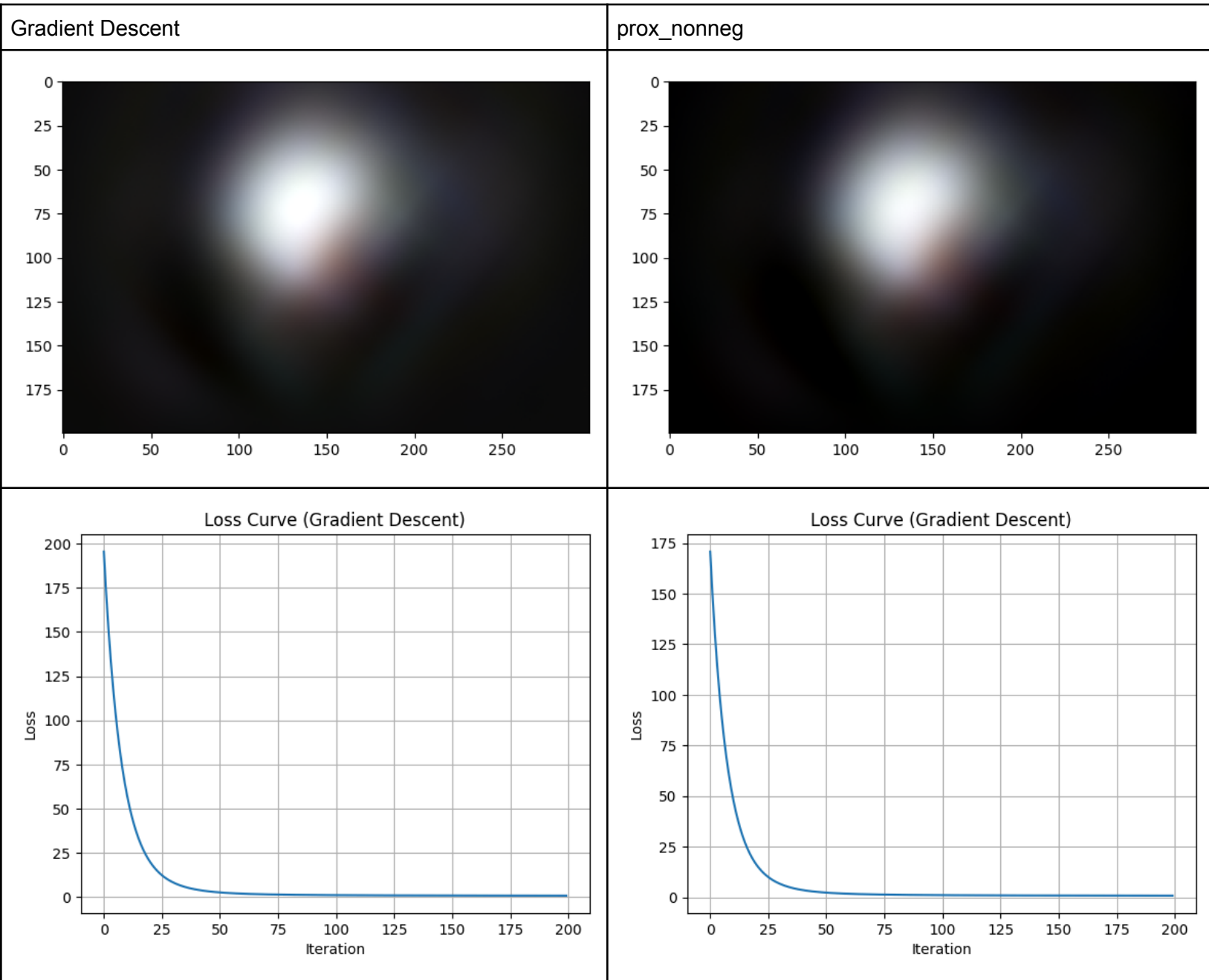
Output: $\mathbf{x}_* \leftarrow \mathbf{x}_k$

Implement FISTA with a non-negativity constraint and try it out on your measurement. Make sure to plot the loss as a function of iteration. You should find that FISTA converges faster than ISTA. If it does not, check your code.

Note: we refer to the proximal method as ISTA and the accelerated proximal method as FISTA here for convenience, but these are generally used to refer to proximal algorithms with an L1 prior rather than a non-negativity constraint in the literature.

Problem 1.1: Fill in `ista_update` in `inverse_solver.py`. It should use `self.prox()` to apply the proximal operator after the gradient step.

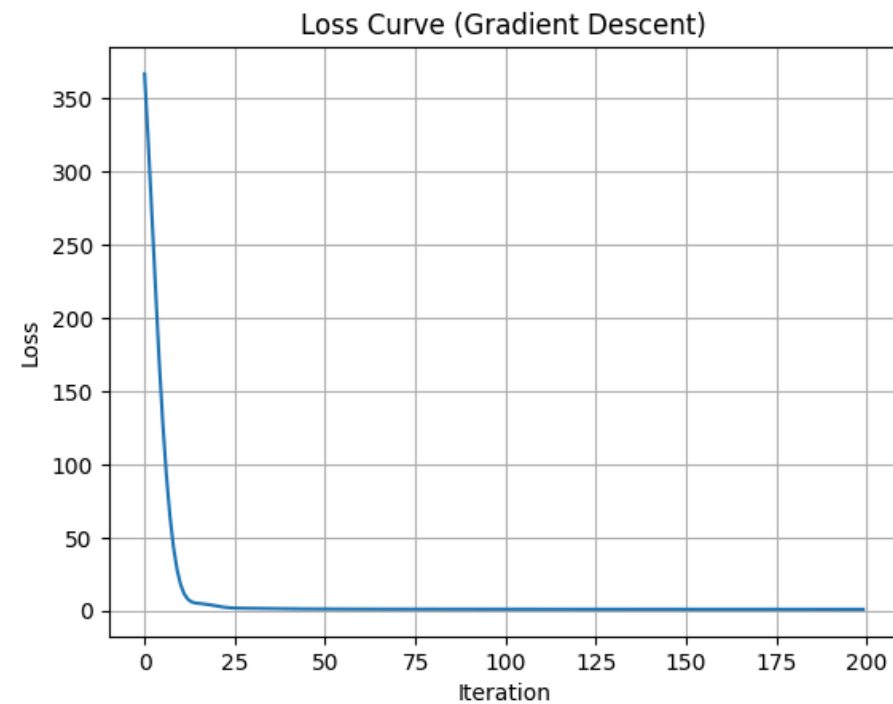
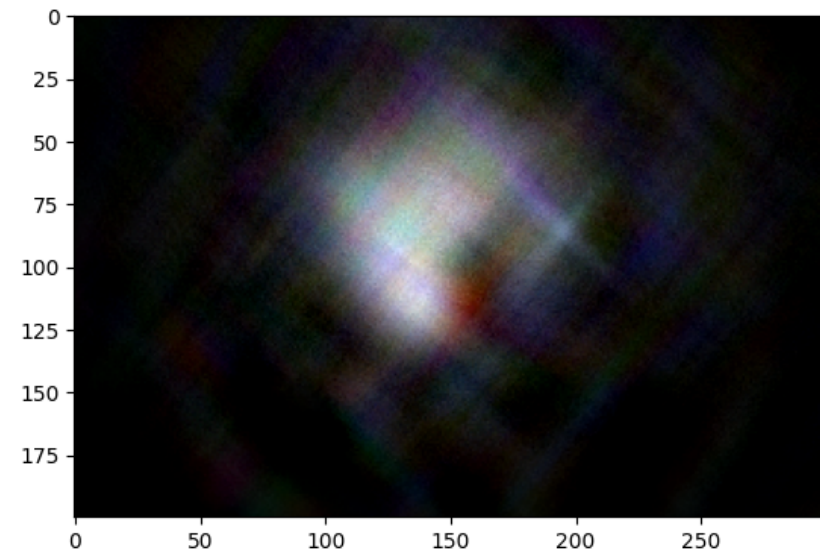
Run ISTA with `prox_fn=prox_nonneg` on your DiffuserCam image for 200 iterations. Display the reconstructed image and plot the loss curve. Compare the result to your gradient descent reconstruction from Lab 7—how does the non-negativity constraint affect image quality?



The non-negativity constraint appears to help lessen the ringing artifact that occurs when using Diffusercam.

Problem 1.2: Fill in `fista_update` in `inverse_solver.py`. It should use `self.prox()` to apply the proximal operator after the gradient step.

Run FISTA with `prox_fn=prox_nonneg` on your DiffuserCam image for 200 iterations. Plot the final image estimates for gradient descent, ISTA, and FISTA side by side. Plot the loss as a function of iteration for all three methods on the same axes. Describe the difference in convergence rates.



FISTA appears to have a much steeper initial drop compared to the other two loss curves. ISTA takes a more gradual decline to converge near 0, unlike normal gradient descent which appears to converge to a non-zero value.

Problem 1.3: A common stopping criterion is the relative change in loss:

$$\frac{|L(\mathbf{x}_k) - L(\mathbf{x}_{k-1})|}{|L(\mathbf{x}_{k-1})|} < \epsilon$$

Using $\epsilon = 10^{-3}$, modify your solver to stop early when this criterion is met (with a maximum of 500 iterations). Run both ISTA and FISTA with `prox_fn=prox_nonneg` on your DiffuserCam image. Report how many iterations each method takes to converge. What speedup does FISTA provide over ISTA?

FISTA converged in 111 iterations.

ISTA converged in 231 iterations.

FISTA provides a $(231/111) = 2.081x$ Speedup

Checkoff

When you're ready, ask a TA to check you off.

2 Native Sparsity

Besides non-negativity, sparsity in some domain is a useful constraint (especially for compressive sensing problems). This is called the lasso problem and has the form:

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

This problem can be interpreted as finding a sparse solution, and is critical in compressive sensing applications. In this setting, $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2$ and $g(\mathbf{x}) = \lambda \|\mathbf{x}\|_1$. Conveniently, this has the following proximal operator:

$$\text{prox}_g = \text{soft}(\mathbf{x}, \lambda) = \text{sign}(\mathbf{x}) \cdot \max(|\mathbf{x}| - \lambda, 0)$$

This is called the soft-thresholding operator.

Implement the soft-thresholding function and try running FISTA with native sparsity on your image. Note, you will need to tune λ to see an effect. A λ that is too big will result in a very sparse solution (almost all zero), whereas a λ that is too small will not have much of an effect. Make sure to plot the loss ($L(\mathbf{x}) = \frac{1}{2} \|\mathbf{y} - \mathbf{A}\mathbf{x}\|_2^2 + \lambda \|\mathbf{x}\|_1$) over iteration.

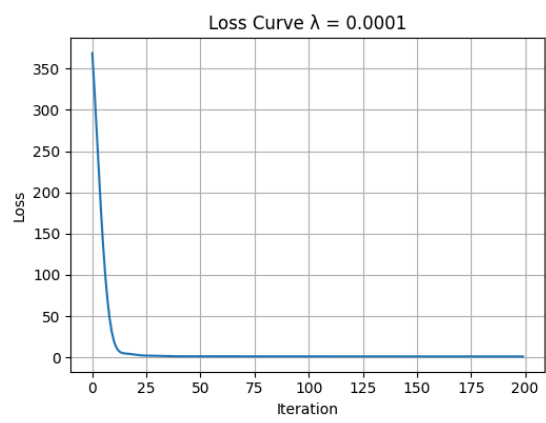
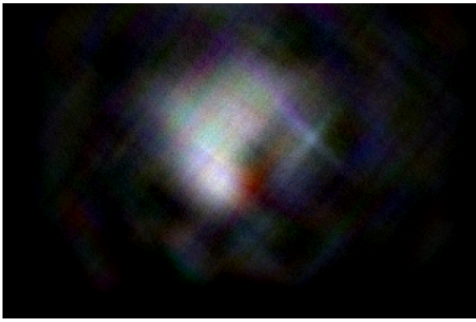
Problem 2.1:

- Fill in `soft_thresh` in `priors.py`.
- Fill in `prox_native` in `priors.py`. This combines non-negativity and L1 sparsity via a proximal average. Run FISTA with `prox_fn=prox_native` on your DiffuserCam image with several values of λ (e.g. 10^{-4} to 1) for 200 iterations each. Display the results. Compare your best native sparsity result to your FISTA + non-negativity reconstruction from Problem 1.2. When does native sparsity improve the reconstruction, and when does it fail?

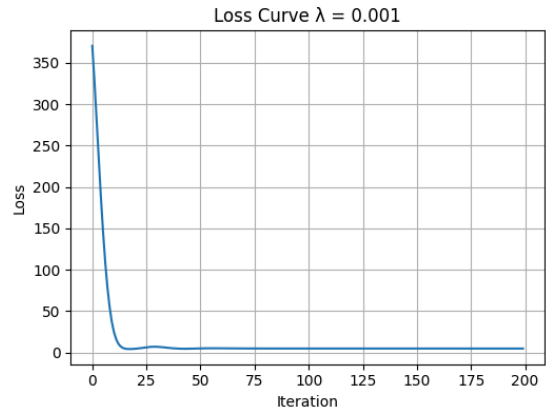
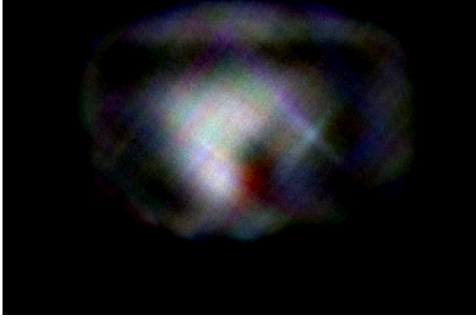
b) For my best native sparsity result of around $\lambda = 1e-3$, it looks a little sharper and less blurry than the result in 1.2, and native sparsity will do poorly for non-sparse images

(like varying regions, landscapes, stuff like that), while it will do better for stuff like an object on a black backgrounds, stars in the sky.

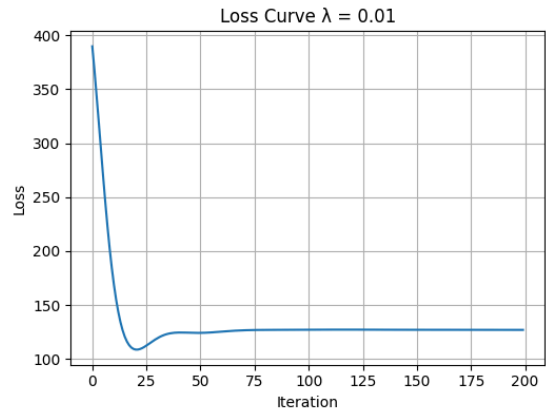
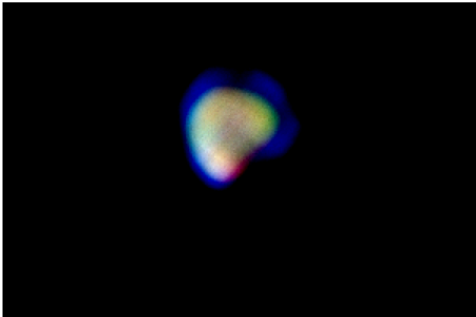
$\lambda = 0.0001$



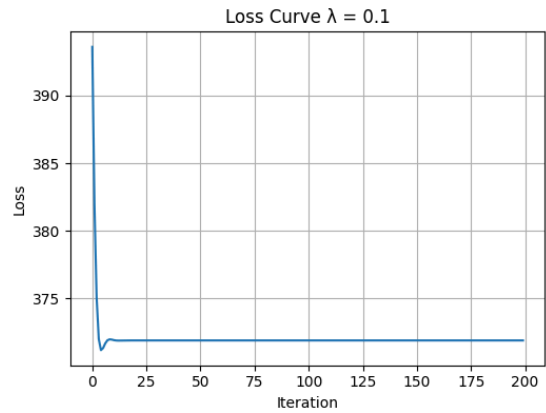
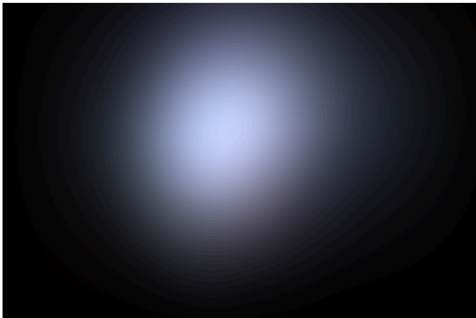
$\lambda = 0.001$



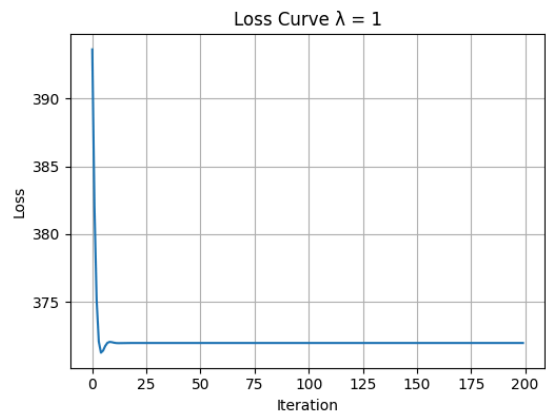
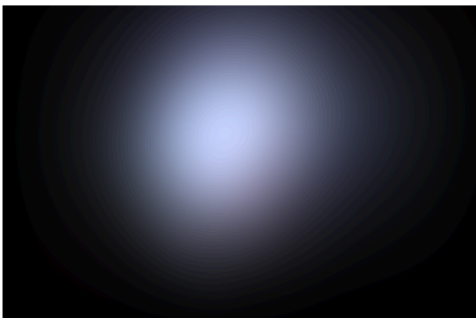
$\lambda = 0.01$



$\lambda = 0.1$



$\lambda = 1$



3 Deriving the Proximal Operator

Problem 3.1: Let

$$g(x) = \begin{cases} 0, & x \geq 0, \\ \infty, & \text{otherwise.} \end{cases}$$

Show that the proximal operator of g is given by

$$\text{prox}_{\lambda g}(v) = \max(v, 0).$$

Hint: Use the definition of the proximal operator and analyze the problem by considering the cases $v \geq 0$ and $v < 0$.

For $\text{prox}_{\lambda g}(v) = \underset{x}{\operatorname{argmin}} \{g(x) + \frac{1}{2\lambda} \|x - v\|_2^2\}$, find the optimal solution by taking the subgradient

- $0 \in \partial g(x) + \frac{1}{\lambda} (x - v)$
- $(v - x) \in \lambda \partial g(x)$

Using the definition of subgradient ($g(y) \geq g(x) + c(y - x)$ for $\forall y$)

If $x > 0$, then $g(x) = 0$, so $g(y) \geq c(y - x)$, which we break down into cases

If $y > x$, then $0 \geq c(y - x)$, which implies that $c \leq 0$

If $0 \leq y < x$, then it forces $0 \geq c(y - x)$ to be $c \geq 0$

If $y < 0$, then $\infty \geq c(y - x)$, which is true for all c

$(x > 0 \text{ implies } \partial g(x) = 0)$

If $x < 0$, then $g(y) \geq \infty + c(y - x)$, which is impossible if $y \geq 0$, so it is undefined

If $x = 0$, then $g(y) \geq c(y)$, so when $y \geq 0$, $c \leq 0$, and when $y < 0$, then $c \leq 0$ is also true

$(x = 0 \text{ implies } \partial g(x) = (-\infty, 0])$

- Using this knowledge, we can derive the operator, if $v \geq 0$, then $\partial g(x) = 0$ when $x_i = v_i$, so $(v - x) = 0$, so $x_i = v_i$

- If $v < 0$, then $\partial g(x) = (-\infty, 0]$ when $x_i = 0$
- Shortening this will get the final expression, $\text{prox}_{\lambda g}(v) = \max(0, v)$

Submission

Submit the following on Gradescope:

4

- A single PDF with answers to all problems, including all requested figures, code, and discussion. Submit this to “Lab 8 Writeup.”
- Your code files (`inverse_solver.py`, `priors.py`, and any scripts). Include a `README.md` explaining how to run your code. Submit this to “Lab 8 Code.”